Linux System Audit Tool

Technical Project Report

Course	Linux Systems Administration — Bash Scripting
Institution	National Higher School of Cyber Security (NSCS)
Academic Year	2025 / 2026
Authors	Souheib Soukeur | Moulai Mostefa Mohamed Adnane
Date	April 26, 2026
Version	1.0 — Final Submission

Abstract

This report presents the design and implementation of a modular Bash-based Linux system audit tool. The tool automates hardware and software information collection, generates reports in multiple formats, and supports automated scheduling via cron and remote monitoring over SSH. It was developed as part of the Systems course at NSCS and demonstrates practical Bash scripting, process management, and network communication skills.

Table of Contents

1. Project Overview

The Linux System Audit Tool is a modular, menu-driven Bash application that automates the collection, formatting, and distribution of system information on Linux machines. Written entirely in Bash 5, it is portable across Debian/Ubuntu-based systems and requires only standard utilities available by default on distributions such as Kali Linux.

The tool is structured around a central entry point (main.sh) that sources six independent modules. Each module can be executed standalone or as part of a fully automated pipeline through run_audit.sh, making the codebase easy to extend and maintain.

1.1 Objectives

- Automate hardware inventory collection: CPU, RAM, disk, GPU, NICs, USB, and motherboard BIOS.
- Collect software state: OS release, kernel version, installed packages, active services, open ports, and user sessions.
- Generate reports in three formats: short plaintext summary, structured JSON, and a detailed full-system report.
- Deliver reports by email using whichever MTA is available on the host.
- Support unattended execution through cron with configurable schedules.
- Enable remote system auditing and live monitoring over SSH, with report transfer via SCP.

1.2 Requirements & Environment

Requirement	Detail
OS	Linux — Debian/Ubuntu family; tested on Kali Linux
Shell	GNU Bash 5.x
Core utilities	lsblk, lspci, lsusb, free, df, ip, ss, systemctl, dpkg, ps, who, uname
Optional	mail / msmtp / sendmail for email; ssh / scp for remote module
Permissions	Standard user; root recommended for DMI/BIOS data

2. Architecture & Project Structure

2.1 Directory Layout

The project is organized as a flat module structure under a single scripts/ root:

```
scripts/
    main.sh                 # Main entry point (interactive menu)
    README.md              # User documentation
    modules/
        hardware_audit.sh     # Part 1 — Hardware collection
        software_audit.sh     # Part 2 — OS & software info
        report_generator.sh   # Part 3 — Report formats
        send_email.sh         # Part 4 — Email dispatch
        setup_cron.sh         # Part 5 — Cron scheduling
        remote_monitor.sh     # Part 6 — SSH monitoring
        run_audit.sh          # Full pipeline orchestrator
        reports/              # Output directory (auto-created)
```

2.2 Module Dependency Overview

main.sh sources the three core modules (hardware_audit.sh, software_audit.sh, report_generator.sh) to import their functions. Options 4, 5, and 6 launch their respective modules as subprocesses using bash, isolating their interactive sub-menus in child shells. run_audit.sh acts as a non-interactive pipeline that sources all modules and runs them in sequence, writing output to files.

Module	File	Role	Integration
Entry Point	main.sh	Interactive menu; dispatches user choice	sources 3 core modules
Part 1	hardware_audit.sh	CPU, RAM, disk, GPU, NIC, MAC, board, USB	sourced by main.sh
Part 2	software_audit.sh	OS, kernel, packages, users, services, ports	sourced by main.sh
Part 3	report_generator.sh	TXT, JSON, full report generation	sourced by main.sh
Part 4	send_email.sh	Email dispatch via mail/msmtp/sendmail	bash subprocess
Part 5	setup_cron.sh	Cron install/remove/status; run-now option	bash subprocess
Part 6	remote_monitor.sh	SSH audit, SCP transfer, live monitoring loop	bash subprocess
Pipeline	run_audit.sh	Orchestrates all; writes raw + final reports	standalone/cron

Table 1 — Module summary

3. Module Descriptions

3.1 main.sh — Entry Point

main.sh is the sole user-facing entry point. It extends PATH with /usr/sbin and /sbin (required for admin tools in cron contexts), resolves its own directory using BASH_SOURCE[0] so relative paths remain valid regardless of the working directory, then sources the three core modules to import their functions into the current shell session.

A while true loop presents the numbered menu repeatedly until the user selects 0. Options 4, 5, and 6 are launched as bash subprocesses so that exiting their sub-menus does not terminate the parent shell.

3.2 Part 1: hardware_audit.sh — Hardware Audit

This module collects the following hardware data:

- CPU: model name from /proc/cpuinfo, core count via nproc, frequency in MHz.
- Memory: total/available/swap via free -h and swapon.
- Disk: block devices listed with lsblk; usage per mount-point with df -h.
- GPU: PCI device grep for VGA/3D/Display entries using lspci.
- Network: full interface listing with ip addr show (fallback: ifconfig).
- MAC addresses: read directly from /sys/class/net/*/address.
- Motherboard / BIOS: vendor, board name, BIOS version from /sys/class/dmi/id/.
- USB: connected devices via lsusb.

All functions are prefixed audit_ and called by run_hardware_audit(). Missing tools are detected with command -v before invocation, and the script prints a warning instead of crashing.

3.3 Part 2: software_audit.sh — Software Audit

- OS identity: sources /etc/os-release for PRETTY_NAME and VERSION_ID.
- Kernel & architecture: uname -r and uname -m.
- Package count: dpkg -l | grep ^ii | wc -l (RPM fallback included).
- Users: active sessions via who; shell-capable accounts from /etc/passwd.
- Services: running systemd units via systemctl list-units --state=running.
- Processes: top 10 by CPU via ps aux --sort=-%cpu.
- Open ports: ss -tulnp (netstat as fallback).
- Environment: SHELL, USER, HOME variables; active cron jobs.

3.4 Part 3: report_generator.sh — Report Generator

Three output formats are produced, all timestamped (date +%Y%m%d_%H%M%S) and written to modules/reports/ (auto-created with mkdir -p):

Format	Filename pattern	Contents
Short TXT (.md)	short_report_YYYYMMDD_HHMMSS.md	Host, date, user, CPU, cores, RAM, OS, kernel, package count, logged-in users.
Short JSON	short_report_YYYYMMDD_HHMMSS.json	Same fields encoded as a flat JSON object via heredoc. No external tool required.

Format	Filename pattern	Contents
Full TXT (.md)	full_report_YYYYMMDD_HHMMSS.md	Complete hardware + software audit output captured from both run_* functions.

Table 2 — Report output formats

3.5 Part 4: send_email.sh — Email Sender

The email module probes for three MTAs in priority order using command -v: mail (mailutils), msmtp, and sendmail. The subject line is auto-composed from the hostname and current date. If no MTA is found, installation instructions for msmtp are printed instead of failing silently.

3.6 Part 5: setup_cron.sh — Cron Automation

Installs a crontab entry that runs run_audit.sh and appends all output to reports/cron_log.txt. Before adding, it removes any existing entry for the same script to prevent duplicates. Available schedule presets:

Option	Cron Expression	Notes
Daily at 04:00	0 4 * * *	Quiet overnight window, recommended for servers.
Hourly	0 * * * *	High-frequency monitoring.
Every 6 hours	0 */6 * * *	Balanced; good for health checks.
Custom	User-supplied	Accepts any valid cron expression.

Table 3 — Cron schedule presets

3.7 Part 6: remote_monitor.sh — Remote Monitoring

Three operations are available:

- Remote audit: runs a heredoc of audit commands on a remote host via ssh user@host bash -s, collecting hostname, OS, kernel, CPU, RAM, disk, uptime, and user count.
- SCP transfer: copies a locally-generated report to a specified path on a remote server. Exit code $? is checked to confirm success.
- Live monitoring loop: polls the remote host every 10 seconds for uptime, memory, and root filesystem usage. Breaks automatically on SSH failure.

4. Technical Implementation Details

4.1 Shell Design Patterns

Pattern	Description
BASH_SOURCE[0] resolution	Every module computes SCRIPT_DIR using cd "$(dirname "${BASH_SOURCE[0]}")" && pwd. This keeps file paths correct whether the script is sourced or executed directly.
source vs bash	Core audit modules are sourced (shared function namespace). Interactive sub-menus are launched as bash subprocesses so their exit does not affect the parent shell.
command -v guards	Before calling any optional binary, the script tests availability with command -v >/dev/null and provides a graceful fallback or message.
PATH extension	PATH is extended with /usr/sbin and /sbin at startup — critical in cron environments where the default PATH is minimal.
Output redirection & logging	run_audit.sh captures each module's stdout to a raw file and appends timestamped entries to audit_log.txt for traceability.

Table 4 — Key Bash patterns used in the project

4.2 Report Formats

The JSON report is generated with a cat <<EOF heredoc, embedding shell variable expansions directly. This avoids any dependency on jq or Python while producing valid machine-readable output. Integer fields (cpu_cores, installed_packages, logged_in_users) are emitted without quotes to match JSON number semantics.

The full TXT report captures ANSI color codes from the audit functions. If color-free output is needed for archiving, these can be stripped with: sed 's/\x1b\[[0-9;]*m//g'

4.3 Cron Integration

The cron entry generated follows this pattern:

```
0 4 * * *  /bin/bash "/path/to/run_audit.sh" >> "/path/to/cron_log.txt" 2>&1
```

The install function reads the current crontab with crontab -l, removes any prior entry matching the script path via grep -v, appends the new line, and pipes the result back to crontab -. This is a standard atomic-replacement pattern that avoids partial writes.

4.4 SSH & SCP Remote Operations

The remote audit uses bash -s with a here-document piped over SSH, avoiding the need to copy any script to the remote machine. The heredoc delimiter is single-quoted (<<'REMOTE_COMMANDS') to prevent local variable expansion — all $(...) expressions expand on the remote host, not locally.

The live monitoring loop checks the SSH exit code after each poll iteration. On failure it prints a connection-lost message and breaks cleanly, preventing the terminal from hanging on a dropped connection.

5. Testing & Validation

The following test cases were run on Kali Linux:

#	Test Case	Method	Result
1	Hardware audit completeness	Compare output fields to /proc/cpuinfo, /sys, lsblk	PASS
2	Software audit package count	Cross-check dpkg -l count with script output	PASS
3	Short JSON validity	Feed output to python3 -m json.tool	PASS
4	Full report generation	Confirm file in reports/; check non-zero size	PASS
5	Email dispatch (msmtp)	Send report to local test account; verify receipt	PASS
6	Cron installation	Inspect crontab -l after install; confirm single entry	PASS
7	Cron duplicate prevention	Run install twice; verify only one entry remains	PASS
8	Remote audit (SSH)	Target: localhost; compare remote output to local audit	PASS
9	SCP transfer	Verify file arrival at remote /tmp/	PASS
10	Missing tool degradation	Rename lspci; verify warning message, no crash	PASS
11	Direct module execution	Execute each .sh directly; verify standalone output	PASS
12	Cron PATH — sbin tools	Simulate cron env with env -i; confirm lspci resolved	PASS

Table 5 — Test case results

6. Conclusion

The Linux System Audit Tool meets all stated objectives: hardware and software inventory collection, multi-format report generation, email delivery, cron-based automation, and SSH-powered remote monitoring. All 12 test cases passed on the target Kali Linux environment.

The project demonstrates several practical Bash scripting concepts: modular decomposition, portable path resolution, graceful degradation on missing dependencies, atomic crontab updates, and safe heredoc-over-SSH remote execution. The separation between sourced modules and subprocess-invoked modules is a deliberate design choice that keeps the codebase maintainable and each module independently usable.

Possible Extensions

- Strip ANSI codes from full reports before archiving or emailing.
- Add a JSON schema for the short report and validate output with jq.
- Extend package audit to support RPM-based systems (CentOS/RHEL).
- Add HTTPS-based report delivery as an alternative to email.
- Implement delta reports that highlight changes between consecutive audits.

Souheib Soukeur
NSCS 2025/2026

Moulai Mostefa Mohamed Adnane
NSCS 2025/2026